

[CS230] Structuring Machine Learning Projects: Data Mismatch & Train-Dev Set

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-6. ML Strategy Basics - *Completed*

Chapter 7. Error Analysis & Data Mismatch (Current Unit)

- 7.1 Manual Error Analysis - *Completed*
- **7.2 Training vs Dev/Test Distribution Mismatch**
 - * The Web vs Mobile Image Problem
 - * New Dataset: "Train-Dev Set"
 - * Diagnostic Logic Table
 - * Artificial Data Synthesis
- 7.3 Transfer Learning (Next Chapter)

지난 시간 복습 및 연결

지난 시간에 우리는 에러 분석을 통해 '무엇을 고칠지' 우선순위를 정했습니다. 그런데 만약 여러분이 수집한 **20만 장의 '고화질 웹 이미지'**로 학습시켰는데, 정작 서비스할 '**저화질 모바일 이미지**'에서는 모델이 전혀 동작하지 않는다면 어떨까요? 이것은 단순한 과대적합(Variance) 문제가 아닙니다. 공부한 책과 시험 과목이 아예 다른 상황, 즉 **데이터 불일치(Data Mismatch)**입니다. 오늘은 이 까다로운 문제를 해결하는 비밀 무기인 '**Train-Dev Set**'을 배웁니다.

1 Unit Overview

□ 핵심 목표

이 단원은 학습 데이터와 실전 데이터의 분포가 다를 때 발생하는 문제를 해결합니다.

- **진단:** 과대적합(Variance)과 데이터 불일치(Mismatch)를 구분하기 위해 **Train-Dev Set**을 도입합니다.
- **논리:** Train, Train-Dev, Dev 에러 간의 격차(Gap)를 분석하여 모델 상태를 판별합니다.
- **해결:** 인공 데이터 합성(Data Synthesis)을 통해 학습 데이터를 실전 분포에 맞추는 법을 배웁니다.

2 Essential Terminology

데이터셋	구성 (Source)	역할
Train Set	웹 이미지 (20만 장)	모델 파라미터 학습
Train-Dev Set	웹 이미지 (일부 떼어냄)	Variance 진단용 (학습 X)
Dev Set	모바일 이미지 (5천 장)	타겟 성능 검증 및 Mismatch 진단
Test Set	모바일 이미지 (5천 장)	최종 성능 평가

3 Core Concepts: 새로운 진단 도구

3.1 1. The Trap of Standard Split (함정)

웹 이미지 20만 장 + 모바일 이미지 1만 장이 있습니다.

- 나쁜 방법: 전부 섞어서 (Shuffle) 나눈다. → Dev Set의 95%가 웹 이미지가 됨. 실전(모바일) 성능을 측정할 수 없음.
- 좋은 방법: Dev/Test는 오직 모바일 이미지로만 구성한다. (Target 고정). Train에는 웹 이미지를 몰아준다.

3.2 2. Introducing "Train-Dev Set"

위의 '좋은 방법'을 쓰면 Train과 Dev의 분포가 달라집니다. 이때 에러가 높으면 "과대적합 때문인가? 아니면 데이터가 달라서인가?"를 알 수 없습니다. 이를 구분하기 위해 **Train-Dev Set**을 만듭니다.

- 정의: Train Set에서 무작위로 떼어낸 일부 데이터. 학습에는 쓰지 않음.
- 특징: Train Set과 분포가 완벽히 동일함.

4 Deep Dive: The Logic of Diagnosis

이 표가 오늘 강의의 핵심입니다. 에러의 차이(Gap)가 어디서 벌어지는지 봐야 합니다.

□ Mismatch 진단 로직 (닥터 딥러닝의 진단표)

비교 대상	Gap의 의미	진단명 (Diagnosis)
Train vs Human	Avoidable Bias	High Bias (모델이 너무 단순함)
Train-Dev vs Train	Variance Gap	High Variance (과대적합)
Dev vs Train-Dev	Mismatch Gap	Data Mismatch (분포 차이)
Test vs Dev	Overfitting to Dev	Dev Set 과적합

□ 시나리오 분석 (직관적 비유)

상황: Train Error 1%.

- **Case A:** Train-Dev 9%, Dev 10%.
 - Train과 Train-Dev(같은 분포) 사이에서 에러가 폭증했습니다.
 - 진단: **High Variance** (과대적합). 정규화가 필요합니다.
- **Case B:** Train-Dev 1.5%, Dev 10%.
 - 같은 분포(Train-Dev)에서는 잘하는데, 다른 분포(Dev)에 가니 못합니다.
 - 진단: **Data Mismatch**. 데이터를 합성하거나 더 모아야 합니다.

5 Implementation: Automated Diagnosis

자동으로 데이터를 분할하고 문제를 진단하는 코드를 작성합니다.

```
1 import numpy as np
2 from sklearn.model_selection import train_test_split
3
4 class MismatchDiagnostician:
5     def __init__(self, X_web, y_web, X_mobile, y_mobile):
6         # 1. Target(Dev/Test)은 모바일로 고정
7         self.X_dev, self.X_test, self.y_dev, self.y_test = train_test_split(
8             X_mobile, y_mobile, test_size=0.5, random_state=1
9         )
10
11         # 2. Train 은 Set 웹 데이터 사용
12         X_train_all, y_train_all = X_web, y_web
13
14         # 3. 핵심[] Train 에서 Set Train-Dev 을 Set 떼어냄 같은( 분포)
15         self.X_train, self.X_train_dev, self.y_train, self.y_train_dev =
16         train_test_split(
17             X_train_all, y_train_all, test_size=0.02, random_state=1
18         )
19
20     def diagnose(self, train_err, train_dev_err, dev_err):
21         print("--- Diagnosis Report ---")
22         variance_gap = train_dev_err - train_err
23         mismatch_gap = dev_err - train_dev_err
24
25         print(f"Variance Gap (TrainDev - Train): {variance_gap:.2%}")
26         print(f"Mismatch Gap (Dev - TrainDev): {mismatch_gap:.2%}")
27
28         if variance_gap > mismatch_gap:
29             print(">> Diagnosis: High Variance (Overfitting)")
30             print(">> Action: More Data, Regularization, Dropout")
31         else:
32             print(">> Diagnosis: Data Mismatch (Distribution Shift)")
33             print(">> Action: Artificial Data Synthesis, Collect Target Data")
34
35 # --- 실행 ---
36 if __name__ == "__main__":
37     # 더미 데이터 웹( 만5, 모바일 천2)
38     X_w, y_w = np.zeros((50000, 10)), np.zeros(50000)
39     X_m, y_m = np.zeros((2000, 10)), np.zeros(2000)
40
41     doctor = MismatchDiagnostician(X_w, y_w, X_m, y_m)
```

```

42 # 시나리오: Train 1%, Train-Dev 1.5%, Dev 10%
43 doctor.diagnose(0.01, 0.015, 0.10)

```

Listing 1: Data Mismatch Diagnosis Tool

6 FAQ & Pitfalls

Q. Train-Dev Set을 Dev Set에서 떼어내면 안 되나요?

A. 절대 안 됩니다. 그러면 Train-Dev와 Dev의 분포가 같아집니다. Train-Dev는 반드시 **Train Set**의 부분집합이어야 ”학습 데이터 분포에서의 일반화 성능”을 측정할 수 있습니다.

Q. Data Mismatch 해결책인 '데이터 합성'은 어떻게 하나요?

A. 깨끗한 음성 (Train)에 자동차 소음 (Noise)을 섞어서 시끄러운 음성 (Dev와 비슷함)을 만드는 식입니다. 단, 너무 적은 종류의 소음만 반복해서 쓰면 모델이 그 소음 패턴에 과적합될 수 있으니 주의해야 합니다.

다음 단계 (Next Step)

이것으로 앤드류 응 교수의 '머신러닝 프로젝트 구조화 전략' 파트를 마칩니다. 여러분은 이제 단순히 코딩만 하는 엔지니어가 아니라, 프로젝트의 방향을 지휘하는 **전략가(Strategist)**가 되었습니다.

다음 시간부터는 다시 모델링의 세계로 돌아옵니다. 컴퓨터 비전(Computer Vision)의 혁명을 일으킨 **[Convolutional Neural Networks (CNN)]**의 기초부터 심화까지, 이미지 처리의 마법을 수학적으로 파헤쳐 보겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Problem:** 학습 데이터(Web)와 실전 데이터(Mobile)의 분포가 다르면 성능이 떨어진다.
2. **Tool:** Train-Dev Set (Train과 분포는 같으나 학습엔 안 씀).
3. **Logic:** Train-Dev 에러가 낮고 Dev 에러가 높다면 **Data Mismatch**다.
4. **Action:** 인공 데이터 합성(Synthesis) 등을 통해 Train을 Dev스럽게 만들어라.